ALASCA — Architecture logicielles avancées pour les systèmes cyber-physiques autonomiques

© Jacques Malenfant

Master informatique, spécialité STL – UFR 919 Ingénierie

Sorbonne Université
Jacques.Malenfant@lip6.fr

CC BY NC ND

Cours 1

Introduction aux systèmes de contrôle cyber-physiques auto-adaptables

BY NC ND

Description courte de l'UE

Étude des architectures logicielles pour les systèmes de contrôle cyber-physiques et les jumeaux numériques, de leurs concepts, théories, techniques et méthodologies. Architectures à base de composants et fondées sur les services appliquées aux systèmes cyber-physiques, aux jumeaux numériques, à l'informatique autonomique, à la robotique autonome et aux systèmes embarqués et répartis à grande échelle.

cc BY NC ND

Plan

1. Organisation du cours
2. Évolution de l'informatique et théorie des systèmes
3. Auto-adaptabilité et systèmes cyber-physiques
4. Autonomicité et contrôle
5. Architectures logicielles de contrôle cyber-physiques et autonomiques

cc BY NC ND

Généralités

- UE professionnalisante et recherche STL :
  - exige un équilibre entre les aspects conceptuels et pratiques.
- 14 semaines (voir le calendrier du master, onglet M2-STL) :

  Mercredi, 13h45 – 15h45 : 2h cours magistraux.

  Mercredi, 16h00 – 18h00 : 2h de travaux pratiques/dirigés
  - travail sur les projets
  - présentations techniques (outils)
  - audits de projets
  - travail sur les sujets de recherche
- Évaluations :
  - Audit 1 (projets) : 14/10/2026
  - Soutenances mi-semestre : 25/11/2026 (rendu 22/11, minuit)
  - Audit 2 (projets) : 6/01/2027
  - Soutenances finales (projets) : 10/02/2027 (rendu 7/02, minuit)
  - Rapports (sujets de recherche) : 10/02/2027, minuit

cc BY NC ND

Modalités d'évaluation

Deux modalités, selon le parcours :

1. Une modalité parcours professionnel ou recherche empirique :
   - un projet à faire en équipe de 2 à soutenir à la fin de l'UE :
     1. premier audit intermédiaire : 10%
     2. soutenance à mi-semestre (programme) : 30%
     3. second audit intermédiaire : 10%
     4. recette de fin de projet (programme, doc, résultats) : 50%
   - sujets :
     - parcours pro : gestionnaire d'énergie autonome.
     - parcours recherche : architectures et outils pour les systèmes cyber-physiques et jumeaux numériques, extensions au projet.
2. Une modalité parcours recherche conceptuelle et théorique :
   - un travail individuel de recherche bibliographique avec une présentation et un rapport en fin d'UE :
     1. pré-rapport, plan du reste à faire à mi-semestre : 25%
     2. présentation finale : 25%
     3. rapport : 50%

cc BY NC ND

Contenu semaine par semaine I

1. Introduction aux systèmes de contrôle cyber-physiques auto-adaptables
2. Architectures logicielles dynamiquement adaptables
3. Modélisation des systèmes de contrôle cyber-physiques
4. Simulation des systèmes cyber-physiques
5. Simulation modulaire et DEVS
6. Théorie et ingénierie du contrôle
7. De l'entité adaptable — principes généraux et conception des capteurs

BY NC ND

Contenu semaine par semaine II

8. De l'entité adaptable — conception des actionneurs et méthodologie de développement
9. De l'entité de contrôle — principes et conception
10. De l'entité de contrôle — méthodologie et développement
11. De l'entité auto-adaptable
12. Composition, coopération et coordination des entités auto-adaptables
13. Décision et contrôle
14. Méta-contrôle, informatique systémique et émergence

cc BY NC ND

Objectifs pédagogiques du cours 1

- Comprendre les enjeux de l'informatique contemporaine où les logiciels :
  - s'exécutent de plus en plus en *permanence*,
  - en lien direct avec le monde réel,
  - avec lequel ils interagissent de manière autonome (*capteurs, contrôle, actionneurs*),
  - et sont capables de s'auto-adapter.
- Comprendre ce qu'est l'*auto-adaptabilité dynamique* des logiciels, et les besoins auxquels elle répond et ses exigences en termes informatiques.
- Comprendre les notions fondamentales des principales *architectures logicielles auto-adaptables* :
  - les systèmes cyber-physiques,
  - les systèmes autonomiques,
  - les jumeaux numériques,

  leurs caractéristiques ainsi que les connaissances et les savoir-faire mis en jeu.

CC BY NC ND

Plan

1 Organisation du cours

2 Évolution de l'informatique et théorie des systèmes

3 Auto-adaptabilité et systèmes cyber-physiques

4 Autonomicité et contrôle

5 Architectures logicielles de contrôle cyber-physiques et autonomiques

CC BY NC ND

L'informatique aujourd'hui I

- Depuis le WWW, les téléphones « intelligents « et l'internet des objets, tout devient réparti et embarqué :
  - Omniprésence de l'informatique répartie, ouverte et à ressources partagées (*service-oriented computing*, *cloud computing*, . . .), s'exécutant en permanence (24/24, 7/7). . .
  - . . . sous de fortes variations de charge et de conditions d'exécution.
- Et plus récemment, les systèmes informatiques sont de plus en plus connectés au monde physique, en temps réel *i.e.*, systèmes « cyber-physiques » (*cyber-physical systems*) :
  - **Définition** : systèmes intégrant de manière forte informatique et monde physique, s'influençant l'un l'autre, parce que les calculs dépendent du monde physique (ex.: localisation), influent sur lui (ex.: voiture autonome) ou dont la performance dépend (ex.: disponibilité de l'énergie).

  Ex.: développement de l'industrie 4.0+.

cc BY NC ND

L'informatique aujourd'hui II

- Enfin, on assiste à l'émergence d'un nouveau concept, le *jumeau numérique*, littéralement la représention fidèle en numérique d'une entité (physique ou numérique) alliant :
  - modèles réifiés, alignés et manipulables dynamiquement,
  - état réifié de l'entité représentée maintenu constamment à jour par des données issus de capteurs,
  - simulateurs permettant de prédire le comportement futur de l'entité représentée, y compris dans des scénarios testant différentes décisions possibles la concernant,
  - capacité à agir sur l'entité représentée via des actionneurs.

  En gros, le concept de jumeau numérique unifie plusieurs méthodes de conception et d'exploitation venant de l'ingénierie « physique » et logicielle : modélisation (*models@runtime*), simulation, contrôle, réflexion en programmation, auto-adaptabilité dynamique, etc.

CC BY NC ND

Vers les notions plus générale de « système »

- En fait, on constate une grande convergence entre l'informatique et l'ensemble des systèmes technologiques et sociaux-économiques.
  - D'un train à grande vitesse au système de transport à l'échelle d'un pays, tout peut maintenant être vu comme des systèmes cyber-physiques à plus ou moins grande échelle, souvent intégrés les uns aux autres.
- Ceci nous ramène à des théories qui ont été élaborées dès l'émergence des premiers ordinateurs (1945-50) :
  - la *cybernétique*, cherchant à appréhender comment de tels systèmes pourraient être conçus, modélisés, analysés puis construits.
  - la *théorie des systèmes*, dont les principes sont fondés sur l'étude de la structure des systèmes, de leurs flux, de la dynamique de leurs interactions et de leur évolution.

CC BY NC ND

Qu'est-ce qu'un « système » et pourquoi en étudier ?

- La science et l'ingénierie s'y intéressent depuis longtemps pour mieux les *comprendre* et mieux les *construire*.
- La notion de *système* est d'autant plus difficile à cerner qu'elle a été étudiée sous différents angles et dans différents contextes :
  - systèmes *naturels*, comme le système gravitationnel du soleil, de ses planètes et de leurs satellites ;
  - systèmes *sociaux*, comme la ville, avec ses habitants, ses activités, ses transports, etc. ;
  - systèmes *technologiques*, comme un vaisseau spatial, son fonctionnement mécanique et électronique, ses instruments de pilotage, ses instruments scientifiques, etc.
- Les principales raisons d'étudier les systèmes est de savoir :
  - les *configurer i.e.*, choisir les valeurs de certaines de leurs propriétés sur lesquelles on a prise de manière à en influencer d'autres optimisables (performance, consommation, etc.) ;
  - les *contrôler i.e.*, agir sur certaines de leurs propriétés *modifiables dynamiquement* pour en maintenir d'autres sur une trajectoire souhaitée ou optimale.

CC BY NC ND

Fonctions et qualités des systèmes

- On attribue généralement à un système une *fonction*, intentionnelle ou non, qui explique son comportement et associe ses différentes propriétés.
- À cette fonction, on prête des *qualités* qui vont donner lieu à une *évaluation*, ce qui conduit, lorsque leurs valeurs sont comparables, à la recherche des *meilleures solutions*, celles présentant les meilleures qualités, la meilleure évaluation.
  $\Rightarrow$ *En informatique : la performance, la qualité de service, ...*
- Ainsi, pour *comprendre*, *analyser* et *mieux construire* un système, il est très utile :
  - de le *modéliser* de manière à *capturer* ses propriétés essentielles,
  - d'en *inférer* les qualités à partir de ce modèle et de ses propriétés
  - et ainsi *prévoir* le plus fidèlement possible le comportement du système réel, avant même de le construire.

  et pour mieux l'opérer, utiliser ces modèles et les données de capteurs pour maintenir un fonctionnement optimal.

cc BY NC ND

Nature, comportement et propriétés des systèmes

- Les systèmes informatiques exhibent des comportements déclenchés par des *phénomènes discrets* vus comme des *événements* se produisant à des *instants ponctuels* dans le temps :
  - recevoir d'une requête,
  - appeler d'un service,
  - cliquer sur un bouton, etc.

  $\Rightarrow$ Des phénomènes modélisables par des *automates*, changeant d'état aux occurrences d'événements.
- Mais ils exhibent également des évolutions d'état *graduelles* mieux capturés par des *modèles continus* :
  - la durée d'un calcul,
  - le nombre moyen de requêtes par seconde,
  - la consommation d'énergie, etc.

  $\Rightarrow$ Des phénomènes plutôt modélisables par des *équations* mathématiques *continues*.

cc BY NC ND

Les systèmes hybrides

- Les systèmes qui exhibent *à la fois* des évolutions *continues* et *discrètes* sont dits *hybrides*.
- C'est le cas des systèmes cyber-physiques dont la partie informatique exhibe des évolutions discrètes et dont la partie physique exhibe des évolutions continues.
- En mathématiques, le terme « ***systèmes hybrides*** » désigne une famille de modèles (formels) étudiés depuis les annnées 1990, par quatre communautés de recherche :
  - en mathématiques, donc,
  - en informatique théorique,
  - en automatique (théorie et ingénierie du contrôle) et
  - en recherche opérationnelle et théorie de la décision.
- Dans la suite d'ALASCA, nous allons nous appuyer particulièrement sur ces systèmes hybrides pour modéliser les CPS, et qui seront le sujet central du cours 3.

CC BY NC ND

Systèmes complexes

- La théorie des systèmes s'est rapidement intéressée aux systèmes dits *complexes*.
  - La notion de système complexe possède plusieurs acceptions ; en théorie des systèmes, il ne s'agit pas simplement de systèmes *compliqués* à comprendre ou à construire, mais de systèmes qui *défient* les méthodes scientifiques et techniques usuelles.
  - Méthode cartésienne : méthode dérivée des travaux de Descartes où la compréhension d'un tout s'obtient par *induction* à partir de la compréhension *individuelle* de ses parties.
  - Cette méthode ne fonctionne plus pour de tels systèmes.
- Compte tenu des limitations de l'informatique au début de la théorie des systèmes, les seuls systèmes réellement complexes étaient des systèmes naturels (écosystèmes, par exemple) ou sociaux (économie de la France, par exemple).
- L'étude des systèmes complexes a alors pris un axe *analytique* (compréhension de systèmes existants) plutôt que *constructif* (comment construire des systèmes complexes corrects).

cc BY NC ND

La systémique

Systémique : science de l'étude des systèmes *trop complexes* pour être abordés par raisonnement analytique cartésien.

- L'objectif principal de la systémique est de proposer une nouvelle méthode scientifique permettant d'analyser, comprendre et construire des systèmes complexes. Références :
  - « *La méthode* » d'Edgar Morin centrée sur les systèmes naturels/sociaux ;
  - « *Le macroscope* » de Joël de Rosnay centré sur les systèmes biologiques.
- Comme les autres, les systèmes complexes évoluent aussi dans le temps et se caractérisent par des mécanismes diffus visant :
  - au *maintien* de ses structures existantes (homéostasie),
  - au *pilotage* (auto-régulation) de sa « trajectoire » selon des règles prédéterminées, et
  - à son *évolution* par « apprentissage » à partir des interactions avec l'environnement et de leur efficacité (ex.: sélection naturelle).

BY NC ND

Plan

1 Organisation du cours

2 Évolution de l'informatique et théorie des systèmes

3 Auto-adaptabilité et systèmes cyber-physiques

4 Autonomicité et contrôle

5 Architectures logicielles de contrôle cyber-physiques et autonomiques

BY NC ND

Évolution subie versus contrôlée

- Les systèmes évoluent dans le temps sous l'impulsion de deux grands types de phénomènes :
  1. phénomènes *subis*, qui *forcent* une évolution, comme la dégradation due à l'usure ;
  2. phénomènes *contrôlés*, qui permettent d'*agir volontairement* pour les faire évoluer, comme le redémarrage d'un ordinateur pour résoudre une panne transitoire.
- Dans le cas d'un système cyber-physique, amener une évolution volontaire suppose de pouvoir modifier :
  - depuis des paramètres (configuration, quantité de ressources disponibles, etc.),
  - jusqu'à son comportement, à savoir son code,
  - et même la manière de faire ces modifications.
- Par ailleurs, comme souligné en systémique, ces évolutions volontaires devraient pouvoir être impulsées par le système lui-même, de manière *autonome*.

CC BY NC ND

Formes d'adaptabilité I

Les besoins d'adaptation des systèmes découlent des conditions dans lesquelles ils s'exécutent :

1. Un besoin d'adapter les systèmes à leur contexte d'exécution pour assurer la meilleure performance et la meilleure utilisation des ressources possibles (configuration).
   - **Systèmes adaptables** : systèmes qui peuvent être adaptés à leurs différents contextes d'exécution soit par choix des algorithmes ou par des choix de paramètres de déploiement.
2. Si les conditions d'exécution évoluent dynamiquement de manière importante, un besoin de s'adapter en permanence, pendant toute la durée de l'exécution (reconfiguration, pilotage).
   - **Systèmes *dynamiquement* adaptables** : systèmes qui peuvent être adaptés pendant leur exécution.
3. Lorsque les évolutions sont plus rapides et les interventions manuelles plus complexes, un besoin de systèmes sachant s'adapter eux-mêmes de manière autonome.

cc BY NC ND

Formes d'adaptabilité II

- **Systèmes auto-adaptables** : systèmes qui peuvent s'adapter eux-mêmes pendant leur exécution.

4. Concommittamment, le besoin de réagir aux évolutions de l'environnement physique en prenant des décisions et en exerçant sur le système une forme de *contrôle* :
   - **Systèmes de contrôle cyber-physiques** : systèmes cyber-physiques interagissant avec le monde physique grâce à des *capteurs* lui permettant d'en connaître l'état et à des *actionneurs* lui permettant d'agir sur ce dernier (ex.: robotique autonome, pilote automatique, . . .).

   Et en informatique, une déclinaison particulière :
   - **Systèmes autonomiques** : systèmes informatiques auto-adaptables exerçant sur eux-mêmes un contrôle de même nature que celui exercé par les systèmes de contrôle cyber-physiques.

- *Et la combinaison des deux :*

  les ***systèmes de contrôle cyber-physiques autonomiques****.*

CC BY NC ND

Exigences pour l'auto-adaptabilité en informatique

Un système informatique (cyber-physique) auto-adaptable doit avoir :

- une **connaissance de lui-même**, de sa forme, de son contenu, de sa configuration, de son état d'exécution courant, etc. ;
  $\Rightarrow$ réfléxivité, models@runtime, jumeaux numériques, ...
- une **connaissance de son contexte d'exécution**, à tous les niveaux, y compris les propriétés de l'exécution en cours ;
- des **mécanismes lui permettant de se modifier lui-même**, c'est-à-dire de changer sa configuration, les ressources à sa disposition, de même que modifier son architecture et son code ;
- une **capacité à décider quand et comment** il doit s'adapter par rapport à son état courant et celui de son environnement ;
- et enfin, *en poussant à la limite*, une **capacité à modifier sa manière de s'auto-adapter** *i.e.*, l'application « récursive » de la fonction d'auto-adaptabilité à elle-même.

CC BY NC ND

Problématiques induites en développement logiciel

- Représentation du système lui-même :
  - Quelles structures de données manipulables à l'exécution faut-il adopter pour représenter les différents éléments du système ?
  - Comment tenir à jour ces structures de données lorsqu'elles représentent des aspects dynamiques ?
- Implantation de mécanismes d'adaptation :
  - Quelles opérations doivent être rendues disponibles au système pour qu'il puisse s'adapter lui-même ?
  - Comment faire en sorte que l'exécution de ces opérations ne cause pas d'erreurs dans le fonctionnement normal du système ?
- Intégration de capacité d'auto-adaptation :
  - Comment architecturer une fonction de *décision autonome* pour décider quand puis comment adapter le système ?
- Conception modulaire :
  - Adopter une forme de modularité est nécessaire, mais comment *composer* des modules auto-adaptables de manière à ce que leur fonctions de base et leur capacité d'auto-adaptation travaillent « correctement » ensemble ?

cc BY NC ND

Plan

1 Organisation du cours

2 Évolution de l'informatique et théorie des systèmes

3 Auto-adaptabilité et systèmes cyber-physiques

4 Autonomicité et contrôle

5 Architectures logicielles de contrôle cyber-physiques et autonomiques

cc BY NC ND

Typologie des causes d'adaptation à prendre en compte

- Les adaptations sont réalisées pour répondre à des besoins de nouvelles fonctionnalités, de meilleure qualité de service, de pannes à circonvenir ou de réactions imposées par l'évolution de l'environnement.
- *Comment* ces besoins apparaissent-ils et *à quel rythme* ?

Continuum multi-dimensionnel entre

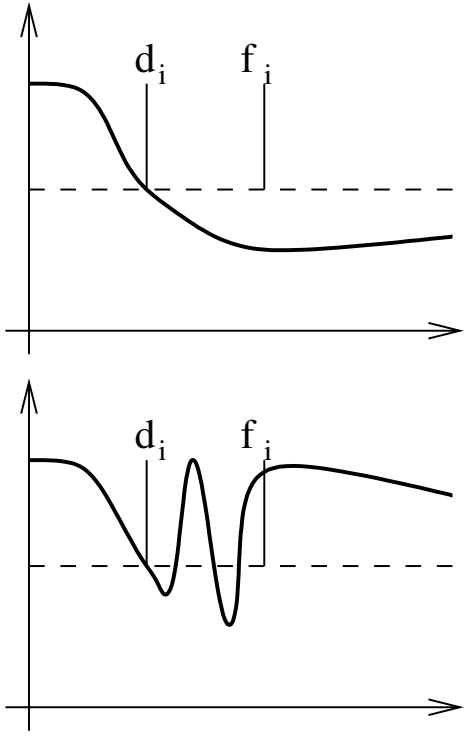
- des changements rares, prédictibles et déterministes
  Ex.: changements planifiés par l'utilisateur dans l'évolution logicielle ou le mode d'opération, niveau de charge de la batterie (+ ou -) d'un portable, ...
- des changements fréquents et aléatoires
  Ex.: évolution de la bande passante du réseau, disponibilité de nouveaux services (ambiants *i.e.*, localisation), etc.

cc BY NC ND

Problème : *le temps* et l'*inertie !*

- Exemple: adaptation à la bande passante d'un réseau sans fil.

- S'adapter prend du **temps** et consomme des **ressources**.
- L'état (interne et contexte) évolue dans le temps (ici, par exemple, la bande passante).
- *Que se passe-t-il si l'état courant n'est plus compatible avec une adaptation qui vient de se terminer ? Que se passe-t-il si les ressources disponibles sont insuffisantes pour adapter complètement le système sur une durée donnée ?*
- Le nouvel état va requérir une nouvelle adaptation.



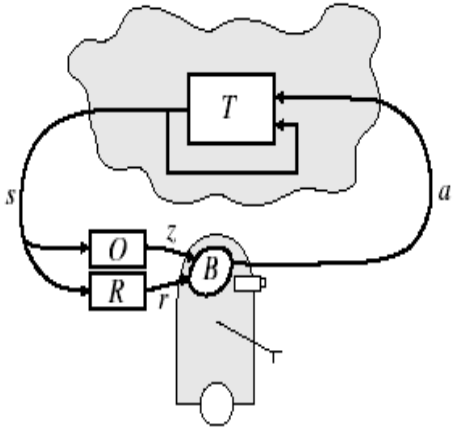
d_i : instant de décision
f_i : fin de l'adaptation lancée à d_i

- Ce processus peut potentiellement mener à une forme d'emballement (« *thrashing* ») : le système s'adapte continuellement, consommant pour cela toutes les ressources disponibles, et ne faisant plus rien d'autre.

cc BY NC ND

Problème bien connu

- Problème d'automatique (contrôle) et de décision.
  - Ex.: pilotage automatique ou l'automatisation des canons anti-aériens pendant la deuxième guerre mondiale.
- Modèle général d'automate de contrôle
  - T : fonction de transition (évolution) de l'environnement selon la décision *a* et l'état *s*
  - O : observation *z* de l'état *s*
  - R : récompense *r* (reward) représentant la *désirabilité* de l'état *s* et le *coût* du contrôle *a*
  - B : comportement (behaviour) c'est-à-dire la décision menant au contrôle *a* exercé



- Problème : équilibrer le gain d'une adaptation *a* par rapport à son coût, ainsi qu'à l'état atteint *s* et à l'évolution possible ensuite.

  Analogie : plus court chemin dans un graphe métrique.
- On doit aussi assurer certaines propriétés sur la trajectoire des états du système, comme éviter l'instabilité et l'emballement.

cc BY NC ND

Auto-adaptation et contrôle = représentation + décision

- Pour s'adapter à la variation des conditions d'exécution et des ressources disponibles en agissant sur un système ou son contexte, il faut savoir décider *quand* et *comment* s'adapter.
- Cela devient un problème à la fois :
  - de décision, au sens de la *théorie de la décision* c'est-à-dire quelle adaptation choisir en fonction d'une analyse coût/bénéfice de l'action choisie versus l'état qu'elle permet d'atteindre mais également les évolutions subséquentes rendues possibles ;
    - Problème de décision séquentielle (approches RO et IA).
  - de contrôle au sens de la *théorie du contrôle*, c'est-à-dire comment gérer son impact sur la dynamique du système étant donné l'*écoulement du temps*, son *inertie intrinsèque* et les *contraintes* posées sur les adaptations possibles.
- Les notions de contrôle seront abordées au cours 4 alors que celles de la décision le seront au cours 13.

cc BY NC ND

Du besoin au logiciel

- Réaliser un système auto-adaptable *correct* et *sûr* est une tâche complexe.
  - Ils sont reconnus comme très difficiles à spécifier, mettre au point, tester, vérifier et valider.
- L'objectif principal de l'UE ALASCA est de chercher à répondre aux questions suivantes :
  - *Que doivent proposer un **modèle à composants** et une **architecture logicielle** pour faciliter ces activités et soutenir la mise en œuvre d'une méthodologie de développement propre aux systèmes de contrôle cyber-physiques autonomiques ?*
  - *Comment les construire **efficacement** tout en s'assurant qu'ils soient **corrects** et **sûrs** ?*
- Pour répondre à ces questions, nous allons d'abord regarder ce qui se fait sous le nom d'*autonomic computing*, que nous traduisons par *informatique autonomique* pour plus de simplicité.

CC BY NC ND

Informatique autonomique : objectifs et concepts

Définition : ensemble des concepts et techniques utilisés pour permettre à un système informatique de gérer lui-même les éléments qui le composent et d'entreprendre automatiquement les actions nécessaires à son fonctionnement optimal, sans intervention humaine.

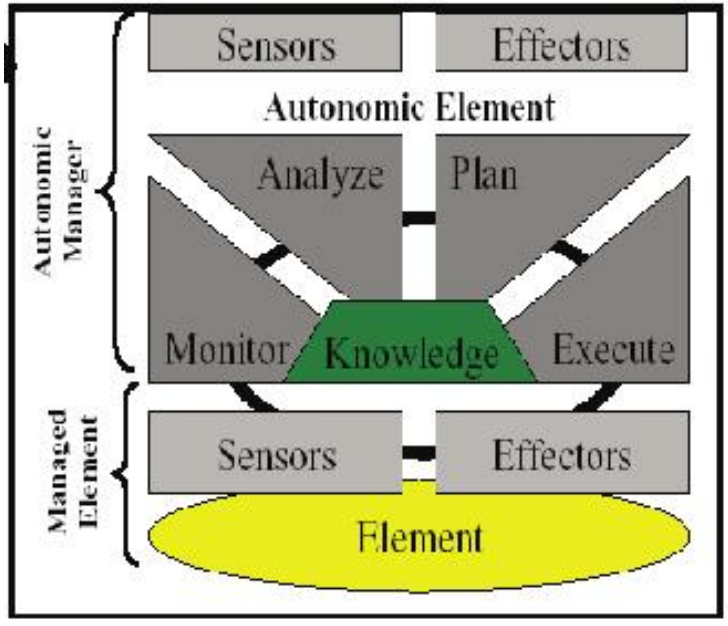
Principaux instigateurs : IBM et ses chercheurs au début des années 2000 qui visaient à

- Mettre l'informatique au service de la gestion des applications pour assurer les propriétés :
  - d'*auto-configurabilité*, d'*auto-optimisabilité*, d'*auto-réparabilité*, d'*auto-protection*, etc.
- Ainsi élever le niveau de fiabilité des services informatiques.
- Et réduire le coût humain de la fonction de gestion des services et applications.

cc BY NC ND

Architecture des éléments autonomiques d'IBM

- La *composition* d'un élément géré et de son contrôleur autonomique forme un *élément autonomique*.



- Surveillance : réception des données via les senseurs.
- Analyse : obtention d'un diagnostic.
- Planification : détermination des actions à prendre.
- Exécution : mise en œuvre du plan via les actionneurs.

- Le contrôleur autonomique est un automate de contrôle du transparent 29 et l'élément géré est l'entité contrôlée.
- La composition des deux forme un système auto-adaptable.

BY NC ND

Lien entre autonomicité et contrôle cyber-physique

- Pendant longtemps, l'informatique autonomique s'est intéressée à l'auto-adaptabilité mais à l'intérieur des frontières « cyber », c'est-à-dire une adaptation à des phénomènes *endogènes* à l'exécution des programmes :
  - consommation mémoire, appels de procédures (nombre, paramètres fréquents, ...), « poids » dynamiques respectifs des processus sur le processeur, etc.
- Avec les systèmes embarqués ou communiquants, elle s'est intéressée de plus en plus à de l'adaptation à des phénomènes *exogènes*, c'est-à-dire de l'environnement :
  - variation de bande passante des réseaux sans fil, la mobilité, la *consommation d'énergie*, etc.
- À cause de leur rythme plus élevé, ces phénomènes exogènes ont fait apparaître les problématiques de contrôle des systèmes cyber-physiques dans les systèmes autonomiques.

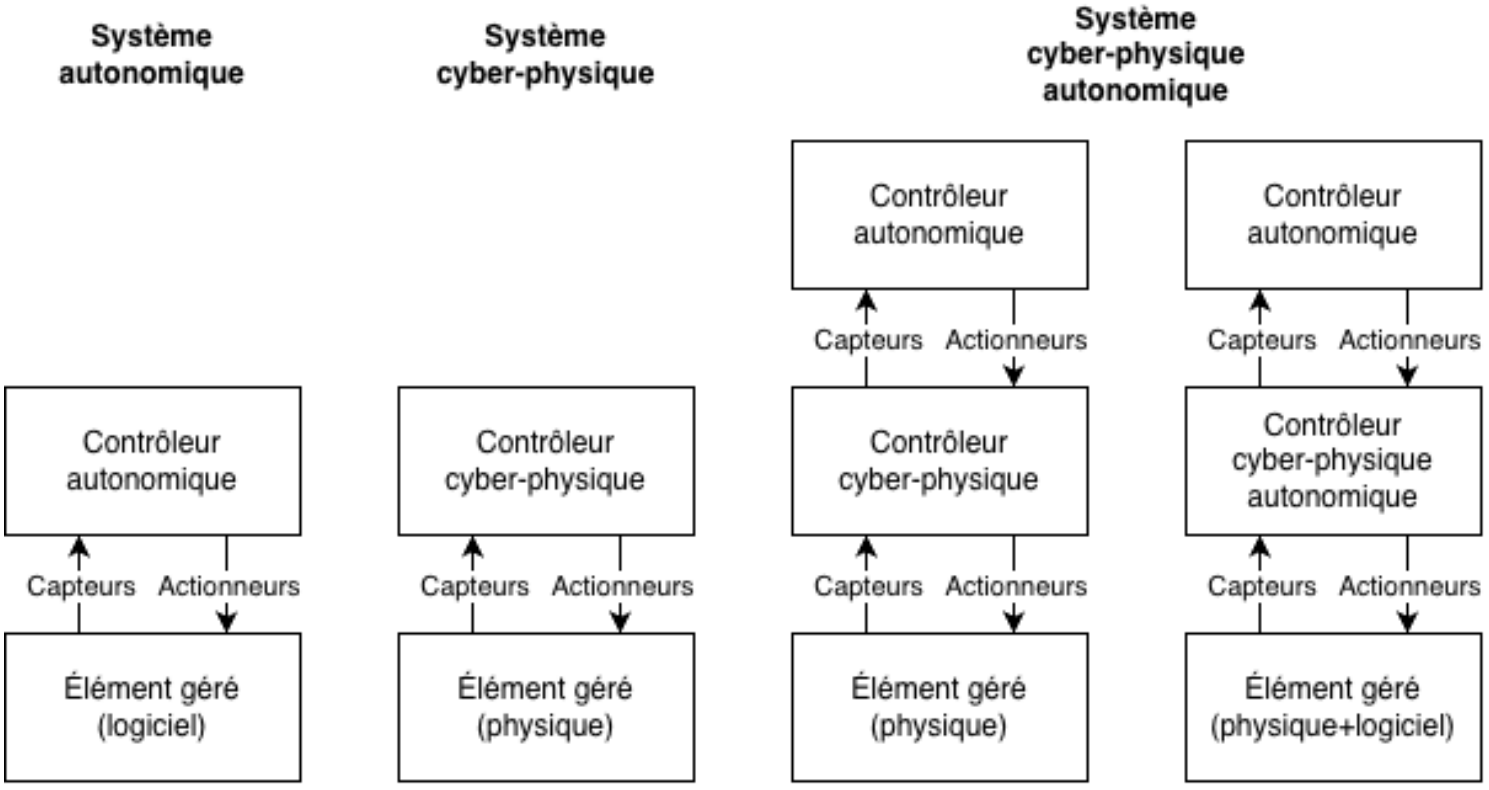
cc BY NC ND

Des systèmes autonomiques et cyber-physiques

- Le patron architectural d'IBM pour l'informatique autonomique :
  - s'inspire des notions de la théorie du contrôle (capteurs, actionneurs, politique, etc.),
  - propose une forme de modularité générique avec des *éléments de contrôle* qui interagissent avec leurs *éléments adaptables* via des capteurs et des actionneurs.
- Un système de contrôle cyber-physique autonomique utilise ainsi les mêmes modèles et techniques de contrôle à deux niveaux :
  1. Au niveau de l'élément logiciel de base (élément adaptable en informatique autonomique) quand il est lui-même la partie « *cyber* » d'un système cyber-physique.
     Ex.: élément logiciel pilotant une voiture autonome
  2. Au niveau de l'élément de contrôle parce qu'il agit sur l'élément logiciel de base de manière similaire à ce que ce dernier fait avec le monde physique.
     Ex.: élément logiciel d'apprentissage adaptant dynamiquement l'élément pilotant la voiture autonome.

CC BY NC ND

Des systèmes autonomiques et cyber-physiques



- Tous les contrôleurs sont soumis et opèrent selon les principes de la théorie du contrôle (automatique).
- Dans les systèmes cyber-physiques (simples), on parle généralement de pilotage plutôt que d'adaptation (quoique).
- Exemple d'élément géré physique+logiciel : voiture autonome.

CC BY NC ND

Disciplines d'appui : un domaine au spectre très large

- Systèmes répartis en réseau
- Systèmes embarqués temps-réel
- **Génie des logiciels à base de composants**
- Modélisation comportementale
- Modélisation stochastique
- Simulation
- Décision
- Théorie du contrôle

Et bien d'autres encore : « informatique-complet » (et plus)...

- En ALASCA, nous allons explorer les contributions de ces domaines, non pour en devenir des spécialistes, mais pour en connaître les principaux concepts et être capables d'en comprendre les implications.

CC BY NC ND

Plan

1 Organisation du cours

2 Évolution de l'informatique et théorie des systèmes

3 Auto-adaptabilité et systèmes cyber-physiques

4 Autonomicité et contrôle

5 Architectures logicielles de contrôle cyber-physiques et autonomiques

cc BY NC ND

Caractère auto-adaptable des architectures logicielles

- Recherche de modularité, deux problèmes fondamentaux :
  1. Comment programmer des *entités modulaires* pour les rendre auto-adaptables ?
     - composant, modèles, connaissance de son propre état et de celui du monde physique, capacité d'auto-adaptation, ...
  2. Comment les composer pour former des systèmes (architectures) auto-adaptables ?
     - compatibilité, composition « parallèle » du code fonctionnel, des modèles, de l'auto-adaptabilité, ...
- Tryptique central en conception modulaire :

  comportements (code, modèles) / interfaces / composition
- L'entité logicielle cyber-physique doit donc être conçue autour :
  1. de modèles fonctionnels, de comportement et d'adaptation ;
  2. d'interfaces explicites exposant et contractualisant fonctions, adaptations, synchronisation et qualité de service ;
  3. de mécanismes de composition couvrant ces mêmes aspects.

cc BY NC ND

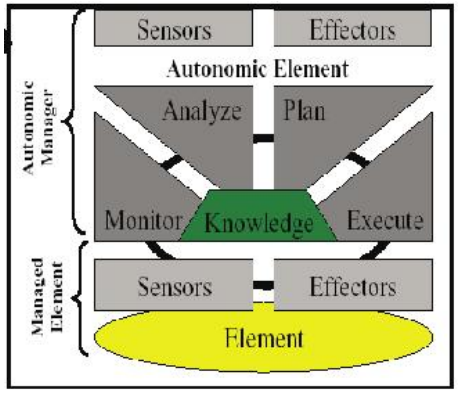
Vers des architectures CCP autonomiques

- Composants de contrôle cyber-physiques autonomiques inspirés :
  - des modèles de composants répartis, temps réel et embarqués ;
  - de l'automatique par l'utilisation de capteurs, d'actionneurs et de contrôleurs en boucle fermée ;
  - de la mesure, de l'instrumentation et de la réflexivité (jumeaux numériques) pour réaliser la « *connaissance de lui-même* » ;
  - des automates et systèmes hybrides pour modéliser le comportement ;
  - des interfaces riches et de la coordination pour la composition,
  - de la simulation modulaire pour le test, la validation et la vérification.
- Intégration de notions de décision : ces composants s'inspirent de l'automatique, de la RO, de l'IA et de la robotique autonome.
- Le contrôle vise trois objectifs :
  1. homéostasie : maintien des équilibres à court terme avec une décision souvent purement réactive ;
  2. contrôle : pilotage autonome selon des règles fixées ;
  3. évolution : introduction de nouvelles règles de contrôle.

cc BY NC ND

De l'élément géré : capteurs et actionneurs

- Les architectures autonomiques à la IBM s'intéressent aux éléments gérés que dans la mesure où ils permettent :
  - d'observer leur structure interne, leur état et leur QdS courants ;
  - de leur appliquer des adaptations.



- Empruntant au vocabulaire de l'automatique, on utilise :
  - le terme *capteur* pour désigner les dispositifs mis en place par l'élément géré pour observer son état courant, et
  - le terme *actionneur* pour désigner l'ensemble des opérations mises en place par l'élément géré permettant de l'adapter dynamiquement.
- En termes d'architecture logicielle,
  - les informations fournies par les capteurs sont rendues visibles et accessibles via des *interfaces capteurs*
  - et les opérations d'adaptation le sont par les *interfaces actionneurs*.

cc BY NC ND

Du contrôleur autonomique : contrôle en boucle fermée I

Quatre grandes étapes :

1. Supervision :
   - opérations permettant de reconstruire une image fidèle de l'état de l'élément géré, de son contexte et de son environnement ;
   - interfaces capteurs de l'élément géré, de même que des sondes sur le contexte et l'environnement ;
   - techniques fondées sur l'historique pour filtrer, débruiter, lisser, simplifier et abstraire les données obtenues pour construire un état utilisable par le modèle de décision et contrôle.
2. Analyse : applique des techniques de corrélation, décision et diagnostic pour
   - déterminer l'état courant à partir des données de supervision,
   - identifier les écarts entre cet état courant et l'état souhaitable,
   - décider s'il est nécessaire d'adapter ou non, et
   - identifier l'*action* d'adaptation et les éventuels paramètres de cette adaptation.

BY NC ND

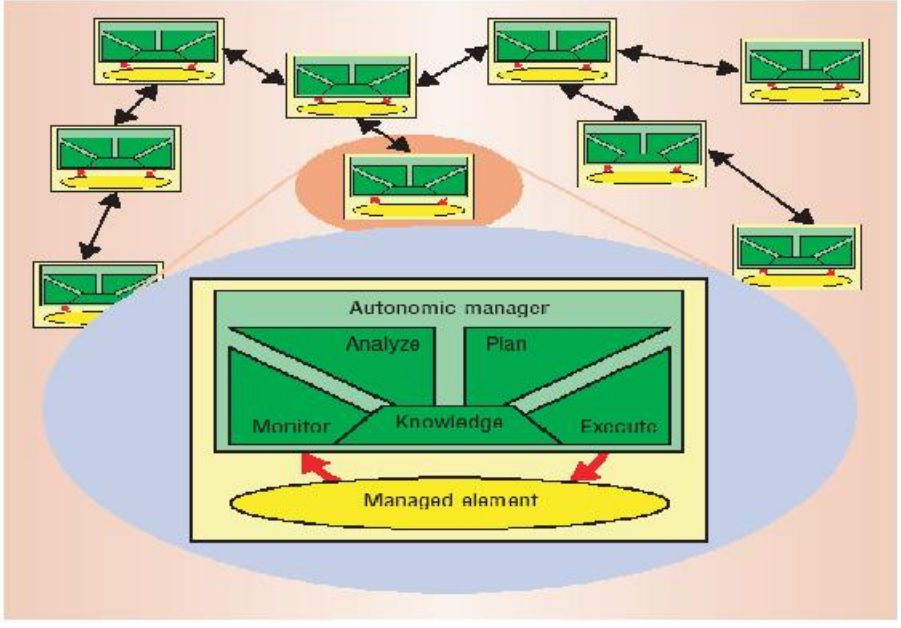
Du contrôleur autonomique : contrôle en boucle fermée II

3. Planification :
   - détermine l'ensemble des *opérations* d'adaptation nécessaires pour réaliser les *actions* d'adaptations choisies, et
   - les organise dans un *plan*, incluant des opérations composites, exécutables sur différentes entités et avec des dépendances temporelles explicites entre elles.
4. Exécution :
   - lance et supervise l'exécution du plan obtenu de la phase de planification ;
   - moteur d'exécution adapté aux types d'opérations élémentaires et composites apparaissant dans le plan, aux actions préparatoires nécessaires (comme l'arrêt momentané d'exécution des éléments gérés) et la synchronisation des actions élémentaires.
   - phase d'exécution est la mise en œuvre du plan par l'ordonnancement, le lancement, le contrôle de réalisation et l'enchaînement des opérations d'adaptation.

cc BY NC ND

Composition des éléments autonomiques

- La plupart des systèmes étant trop complexes et physiquement répartis, ils ne peuvent être contrôlés de manière *monolithique*.
- Il faut donc les voir comme un *assemblage* de composants autonomiques, chacun exerçant son propre contrôle local.



Nouveaux problèmes :

- Connexion entre les éléments autonomiques.
- Coordination des objectifs de contrôle.
- Planification et exécution conjointes des adaptations.

BY NC ND

Genèse des jumeaux numériques

- Domaine très vaste, un peu flou, avec des variantes :
  - Consensus autour d'un représentation de l'état courant de l'entité représentée.
  - Certains ne s'intéressent qu'à la simulation : ils possèdent des capteurs mais pas d'actionneurs.
    - Ex. : jumeau numérique de l'humain.
  - D'autres ne s'intéressent qu'à l'exploitation : ils n'utilisent pas de simulateurs, que de la supervision, du pilotage et du contrôle.
- Selon l'informatique autonomique, un jumeau numérique peut être vu comme un contrôleur autonomique dont la partie connaissance (le K de MAPE-K *i.e., « Knowledge »*) est beaucoup plus détaillée et structurée.
- Selon la réflexion à la B. Smith, un jumeau numérique peut aussi être vu comme une représentation réifiée (numérique) en connexion causale avec son entité réelle (cf. Cours 2).

CC BY NC ND

Architectures des jumeaux numériques I

- Un jumeau numérique « complet » comporte :
  - Un ensemble de miroirs numériques (*digital shadows*) de l'entité, intégrant des traces numériques (*data traces*).
    - Les traces numériques collectent et aggrègent les données capteurs représentant l'état de l'entité dans le temps.
    - Les miroirs numériques offrent des vues sur l'entité à travers des services permettant d'accéder aux données et de les analyser.
  - Un ensemble de modèles multi-physiques et logiciels représentant l'entité réifiés numériquement :
    - modèles d'ingénierie : modèles formels de comportement au sens de la théorie des systèmes (systèmes hybrides) et de la structure conceptuelle de l'entité (SysML) ;
    - modèles de simulation : modèles de comportement exécutables ;
    - modèles 3D : représentation de la forme de l'entité (CAO) ;
    - modèles réflexifs : représentation du code et l'état d'exécution si l'entité en comporte (UML, ASA – ou AST, pile d'exécution, etc.) ;
    - modèles d'auto-adaptation : modèles de variantes, de contrôle, de décision, etc.

CC BY NC ND

Architectures des jumeaux numériques II

- Une IHM permettant de piloter l'entité.
- Un contrôleur de l'entité au sens de l'informatique autonomique.

- Un ensemble de services offerts pour :
  - superviser, conduire des analyses, prédire, trouver des configurations optimales, etc. ;
  - configurer et piloter l'entité en agissant « manuellement » sur les actionneurs via une IHM ;
  - contrôler l'entité au sens des systèmes cyber-physiques.
- D'un point de vue logiciel :
  - Les traces sont connectées à l'entité représentée via des interfaces capteurs.
  - Les miroirs composent les traces et offrent leurs services via des interfaces d'accès et d'analyse.
  - L'IHM de pilotage et le contrôleur autonomique sont connectés à l'entité via les interfaces capteurs et actionneurs.
  - Le contrôleur peut utliser les services d'analyse, de prédiction, de simulation et de décision de son jumeau numérique.

CC BY NC ND

Auto-adaptabilité et composabilité

- Les systèmes sont généralement constitué de plusieurs entités.
  - La recherche de modularité pousse à décomposer puis à devoir composer plusieurs entités auto-adaptables.
    - Composition de contrôleurs autonomiques.
    - Composition (association ou partie-de) de jumeaux numériques.
  - Informatiquement, se limite à interconnecter les contrôleurs ou les jumeaux avec de bonnes interfaces.
  - En réalité, un problème sous-jacent beaucoup plus complexe est la compatibilité des comportements, problème toujours ouvert.
- L'informatique autonomique vise à l'auto-adaptabilité des systèmes informatiques, or un contrôleur autonomique ou un jumeau numérique sont des systèmes informatiques !
  - Méta-contrôleur : contrôleur du contrôleur...
  - Jumeau numérique du jumeau numérique, pourquoi pas ?
  - Attention aux propriétés du contrôle et à la régression infinie potentielle...

cc BY NC ND

Architectures logicielles des domaines connexes I

- Plusieurs architectures autonomiques ont été développées, soit en suivant le plan architectural d'IBM, soit en suivant des variantes de ce dernier.
- Mais d'autres domaines de l'informatique se posent des problèmes similaires et fournissent des sources d'inspiration et de coopération intéressantes.
- Dans les systèmes temps réels embarqués répartis où des actions doivent être prises en réaction à des événements ou à l'évolution de l'état d'un système :
  - architectures synchrones,
  - architectures réparties de type GALS (*globalement asynchrones, localement synchrones*),
  - architectures asservies par le temps (*time-triggered*).
- Architecture de contrôle pour la robotique autonome, où le robot se contrôle lui-même en boucle fermée et peut se coordonner avec d'autres robots :

cc BY NC ND

Architectures logicielles des domaines connexes II

  - architectures réactives,
  - architectures délibératives,
  - architectures hybrides, délibératives/réactives.
- Dans le domaine des systèmes d'information en réseau, les systèmes de traitement d'événements complexes (*complex event processing, CEP*) cherchent à capturer, identifier et corréler des événements spatialement et temporellement répartis pour déclencher des actions.
- Dans le domaine des systèmes multi-agents où s'exercent beaucoup de coordination, de décision collective et d'émergence de comportements globaux, mais généralement sans exigence de temps réel.

cc BY NC ND

Récapitulons...

1. L'informatique aujourd'hui repose de plus en plus sur des logiciels s'exécutant en permanence dans des contextes d'exécution très variables, ce qui impose de les **adapter dynamiquement**.
2. Historiquement, les disciplines scientifiques de la **cybernétique**, de la **théorie des systèmes** et de la **systémique** ont étudié les systèmes ayant une capacité d'auto-adaptation, ce qui permet de faire le lien avec l'informatique par les systèmes cyber-physiques.
3. L'***auto-adaptabilité dynamique*** vise à utiliser la puissance de l'informatique au service de l'adaptabilité dynamique en construisant des logiciels capables de s'adapter eux-mêmes.
4. L'objectif de l'UE est de ***comprendre les problèmes*** liés à la conception et l'implantation de tels logiciels, et de former des ingénieur informatique ***capables de coopérer avec des spécialistes*** des autres disciplines nécessaires pour mener à bien des projets de développement de logiciels auto-adaptables.

CC BY NC ND

Pour aller plus loin : sélection de lectures recommandées

- *La méthode*, Edgar Morin (différentes éditions, en 5 volumes initialement regroupés en deux dans des éditions récentes ; attention : long et érudit).
- *Le macroscope*, J. de Rosnay, Seuil, 1975 (vulgarisation scientifique très accessible).
- *The Vision of Autonomic Computing*, J.O. Kephart et D.M. Chess, Computer, 36, pp. 41–50.
- *Autonomic Computing*, P. Lalanda, J. McCann et A. Diaconescu, Springer-Verlag, 2013, chapitres 1 à 4.
- *An Architectural Blueprint for Autonomic Computing*, IBM, 2006.
- *Autonomic Computing — Concepts, Infrastructure and Applications*, M. Parashar et S. Hariri, CRC Press, 2007, chapitres 1, 3 et 5.

BY NC ND